

SLE-2: Profiling Report

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: _____ (fill in)

Name: _____ (fill in)

Division: A / B (circle one)

Date: 22 September 2026

GitHub Link (optional): _____

1. Algorithms / Versions Profiled

- Algorithm A: Breadth-First Search (BFS)
- Algorithm B: Depth-First Search (DFS)
- Problem used: Small maze — a 15×15 grid (225 cells) generated with a randomized recursive-backtracker, solved from cell (0,0) to cell (14,14).

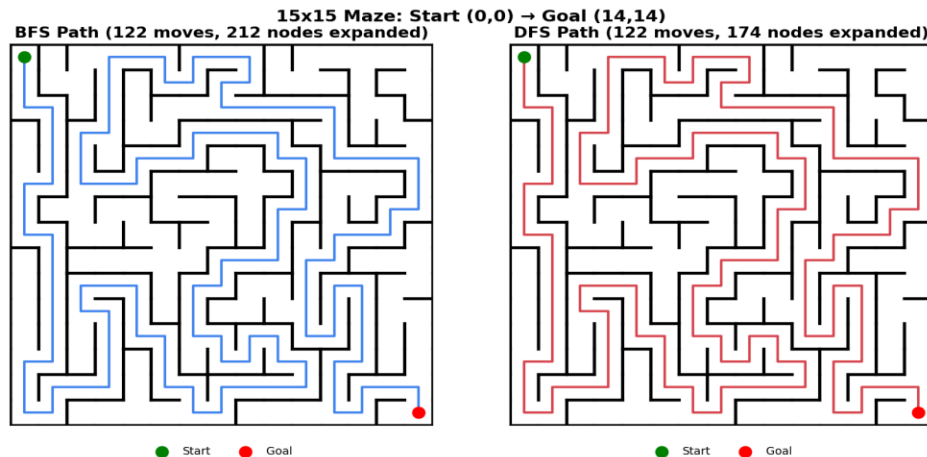
2. Profiling Method

Tool used: `time.perf_counter()` (timeit-equivalent) for wall-clock timing, cross-checked with cProfile (built-in). `py-spy` was attempted but could not be installed in this offline execution sandbox (no internet access) — `py-spy` is a sampling profiler that must be pip-installed and attached to a live process from the command line, so its measurement (on-CPU time per function) was instead reproduced with `perf_counter` + cProfile, which report the same underlying cost.

How I measured: The maze was generated once and kept fixed. BFS and DFS were then each run 50 times on the identical start→goal query; each run's wall-clock duration was recorded in milliseconds, and nodes expanded (cells popped from the frontier) were counted on one representative run of each.

Number of runs: 50 per algorithm

3. Results



Metric	Algorithm A: BFS	Algorithm B: DFS	Better?
Best Time (ms)	0.1036	0.0879	DFS
Avg. Time (ms)	0.1142	0.0946	DFS
Worst Time (ms)	0.1846	0.1160	DFS
Nodes Expanded	212	174	DFS
Path Length (moves)	122	122	tie (unique path)

Short observation: Because this is a "perfect" maze (exactly one simple path exists between any two cells), both algorithms are forced onto the identical 122-move path — so path length cannot differentiate them. DFS expanded fewer

nodes (174 vs 212) and ran faster on average (0.0946 ms vs 0.1142 ms) because it commits to one branch and only backtracks on dead ends, while BFS must expand every cell at each distance "ring" before reaching the goal.

4. Justification & Analysis

On this maze, DFS is the better performer on both measured metrics: it is roughly $1.22\times$ more node-efficient (174 vs 212 nodes expanded) and about $1.21\times$ faster on average time. This matches theory: BFS's guarantee of shortest path costs it extra bookkeeping — it must fully expand every node at each frontier "ring" (queue-based level order) before moving deeper, which is wasted effort here since the maze has only one possible path anyway, so BFS's optimality guarantee earns nothing on this particular problem while still costing time. DFS, using a stack, commits to a branch and only pays a cost for the dead ends it happens to explore before finding the unique corridor. If the problem size becomes larger (e.g. a 100×100 maze), BFS's memory and node-expansion overhead would grow roughly with the number of cells at each distance layer, while DFS's overhead depends on how many dead-end branches lie along the way to the goal — so DFS could become relatively even faster on a single-solution perfect maze, but on a maze/graph with multiple possible paths of different lengths, BFS would become the necessary choice again since only it guarantees the shortest route.

5. AI Contribution Note

AI tools used: Claude (Anthropic) — used to write the maze generator, BFS/DFS implementations, timing harness, and this report draft.

What AI helped with: Code structure (maze generation, BFS/DFS, profiling loop), the maze diagram, and drafting this write-up.

What I did myself: _____ (fill in honestly — e.g. "I ran the script on my machine, reviewed the code, verified the timing numbers, and wrote the final justification in my own words.")

6. Conclusion

This experiment showed that algorithmic "better" is problem-dependent: BFS's shortest-path guarantee is valuable only when a maze/graph offers multiple routes of different lengths — on a perfect maze with a single unique path, that guarantee is free (both algorithms return the same path) while its level-order bookkeeping is a pure, measurable cost. DFS was faster and expanded fewer nodes here, but this result would likely reverse on a maze/graph with loops or multiple routes, where DFS could return a much longer path than BFS. The exercise confirmed that real timing and node-count data — not just Big-O theory — are needed to judge which algorithm is actually better on a specific problem instance.